Chinchilla: Compute-Optimal Scaling.

Given a fixed compute budget, how big should a model be and how much data should it see? Kaplan et al. (2020) said "mostly grow the model." Chinchilla (Hoffmann et al., 2022) re-ran the experiment with the data axis free and found the opposite: *parameters and tokens should grow together*, roughly 20 tokens per parameter. Every number below — the minimisation, the $\sqrt{C}$ exponents, and the GPT-3-vs-Chinchilla contrast — is computed in full, nothing summarised or deferred.

THE EQUATION

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \qquad \text{subject to} \qquad C \approx 6ND$$

Worked example. GPT-3 (N=175e9, D=300e9, $\approx$ 1.7 tok/param) versus Chinchilla (N=70e9, D=1.4e12, 20 tok/-param) at comparable compute; plus a 20:1 allocation table.

Topic 1 of 2 in this module · companion to the course page · v1.0

Read top to bottom: each step feeds the next.

Contents

1 The claim: a parametric loss law

Empirically, the test cross-entropy loss of a transformer language model is captured remarkably well by a power law in two variables — the number of parameters N and the number of training tokens D:

$$\boxed{L(N, D) \;=\; E \;+\; \frac{A}{N^{\alpha}} \;+\; \frac{B}{D^{\beta}}} \tag{1}$$

Each term has a clean reading. E is the *irreducible* loss — the entropy of natural language itself, which no model can beat. A/N^{α} is the penalty for being too small to represent the data; it vanishes as $N \to \infty$. B/D^{β} is the penalty for not having seen enough data; it vanishes as $D \to \infty$. We use the Hoffmann et al. "Approach 3" fit:

Symbol	Meaning	Value / shape
N	model parameters	scalar (count)
D	training tokens	scalar (count)
L	test loss (nats/token)	scalar
E	irreducible loss (entropy floor)	1.69
A, α	size term: scale, exponent	406.4, 0.34
B, β	data term: scale, exponent	410.7, 0.28
C	training compute (FLOPs)	$C \approx 6ND$

Loss is a tug-of-war between two starvations. A small model starves on *capacity* (the A/N^{α} term stays large); a model trained on too little data starves on *experience* (the B/D^{β} term stays large). With a fixed compute budget you can pay down only one debt at the expense of the other. Compute-optimal training is the balance point where a marginal FLOP buys equal loss reduction whether you spend it on parameters or on tokens.

2 The constraint: why compute is $6ND$

Training compute is dominated by the matrix multiplies in forward and backward passes. A forward pass touches each parameter once per token with one multiply and one add (2 FLOPs), and the backward pass costs about twice the forward, giving ≈ 6 FLOPs per parameter per token. Multiplying by N parameters and D tokens:

$$\boxed{C \;\approx\; 6\,N\,D} \tag{2}$$

(The $6ND$ estimate is derived from scratch in Topic 2.) The constraint is what makes the problem interesting: you cannot grow N and D freely; their *product* is pinned by your budget.

3 The derivation: minimise loss at fixed compute

3.1 Substitute the constraint

Fix C. The constraint $C = 6ND$ lets us eliminate D:

$$D = \frac{C}{6N}.$$

Substitute into Eq. (1) to get loss as a function of N alone:

$$L(N) = E + \frac{A}{N^{\alpha}} + B\left(\frac{6N}{C}\right)^{\beta} = E + A\,N^{-\alpha} + B\,6^{\beta}\,C^{-\beta}\,N^{\beta}.$$

The size term *falls* with N; the data term now *rises* with N (because spending the budget on parameters leaves fewer tokens). The minimum is interior.

3.2 Differentiate and set to zero

$$\frac{dL}{dN} = -\alpha A\,N^{-\alpha-1} + \beta B\,6^{\beta}C^{-\beta}\,N^{\beta-1} \;\stackrel{!}{=}\; 0.$$

Move one term across and collect powers of N:

$$\alpha A\,N^{-\alpha-1} = \beta B\,6^{\beta}C^{-\beta}\,N^{\beta-1} \;\Longrightarrow\; N^{-(\alpha+\beta)} = \frac{\beta B\,6^{\beta}}{\alpha A}\,C^{-\beta}.$$

Solving for the optimal parameter count N_{opt} and, via the constraint, the optimal token count D_{opt}:

$$\boxed{N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}, \qquad D_{\text{opt}} \propto C^{\alpha/(\alpha+\beta)}} \tag{3}$$

With the fitted $\alpha = 0.34,\ \beta = 0.28$ the exponents are

$$\frac{\beta}{\alpha+\beta} = \frac{0.28}{0.62} = 0.452, \qquad \frac{\alpha}{\alpha+\beta} = \frac{0.34}{0.62} = 0.548.$$

Both sit close to $\frac{1}{2}$: *parameters and data each scale as roughly* $\sqrt{C}$. When the two exponents α, β are equal the result is *exactly* $C^{0.5}$ for each, and the tokens-per-parameter ratio $D_{\text{opt}}/N_{\text{opt}}$ becomes a constant independent of C — the empirical "20 tokens per parameter" rule.

Because the two exponents are near-equal, N and D should grow in lockstep — each as $\approx C^{0.5}$. Double the compute and you should make the model $\sqrt{2} \approx 1.41\times$ bigger *and* train on $\sqrt{2}\times$ more tokens, holding the ratio near 20:1. This corrects Kaplan et al. (2020), whose held-fixed data schedule made the data exponent look small and led to the advice "grow N much faster than D."

4 Worked example: GPT-3 versus Chinchilla

GPT-3 and Chinchilla were trained at comparable compute but with opposite allocation philosophies. Compute $C = 6ND$ and the ratio D/N for each.

$$\textbf{GPT-3:}\quad N = 175\text{e}9,\ D = 300\text{e}9 \ \Rightarrow\ \frac{D}{N} = \frac{300}{175} = 1.71 \text{ tok/param},$$

$$C = 6(175\text{e}9)(300\text{e}9) = 3.150 \times 10^{23} \text{ FLOPs}.$$

$$\textbf{Chinchilla:}\quad N = 70\text{e}9,\ D = 1.4\text{e}12 \ \Rightarrow\ \frac{D}{N} = \frac{1400}{70} = 20.0 \text{ tok/param},$$

$$C = 6(70\text{e}9)(1.4\text{e}12) = 5.880 \times 10^{23} \text{ FLOPs}.$$

At a similar budget Chinchilla is $2.5\times$ *smaller* yet trained on $4.7\times$ more tokens. DeepMind reported it *beats* GPT-3 on downstream tasks — direct evidence GPT-3 was *undertrained*: it spent its FLOPs on parameters it could not feed.

Model	N	D (tokens)	tok/param	$C = 6ND$ (FLOPs)
GPT-3	175e9	300e9	1.71	3.150×10^{23}
Chinchilla	70e9	1.4e12	20.0	5.880×10^{23}

The heatmap shades the tokens-per-parameter ratio (capped at `indigo!80` for Chinchilla); GPT-3's bar is barely visible — a starving giant.

4.1 The 20:1 allocation table

Using the rule $D_{\text{opt}} = 20N$, the compute-optimal token count and resulting budget for three representative model sizes:

N	$D_{\text{opt}} = 20N$	tok/param	$C = 6ND$ (FLOPs)
1e9	2.0×10^{10}	20	1.200×10^{20}
7e9	1.4×10^{11}	20	5.880×10^{21}
70e9	1.4×10^{12}	20	5.880×10^{23}

The bottom row reproduces Chinchilla's exact allocation — the 70B/1.4T pairing is the 20:1 rule evaluated at $N = 70\text{B}$.

5 Properties / consequences that matter

1. **Equal-exponent balance.** At the optimum the marginal loss-reduction per FLOP is equal across the N and D axes; this is exactly the condition $dL/dN = 0$ after substituting the budget.
2. **Square-root scaling.** $N_{\text{opt}} \approx D_{\text{opt}} \approx C^{0.5}$. A 100× compute increase calls for ≈ 10× more parameters *and* 10× more tokens — not 100× on either alone.
3. **Constant ratio.** When $\alpha \approx \beta$, $D_{\text{opt}}/N_{\text{opt}}$ is budget-independent: a single number (≈ 20) summarises the whole frontier.
4. **GPT-3 was undertrained.** At 1.7 tok/param it sat far from the 20:1 frontier; a smaller model on more data at the same compute would have been better.

> The 20:1 rule is *training-optimal*, not *inference-optimal*. It minimises the loss achievable for a fixed *training* budget — it ignores serving cost. A model you will query billions of times wants the smallest N that hits a target quality, because every inference call costs $\propto N$ forever. So heavily-served models are deliberately *over-trained*: pushed far past 20 tok/param onto a flatter part of the loss curve to shrink N. This is the LLaMA philosophy (e.g. a 7B model on $\sim$ 1T+ tokens, $\sim$ 140:1), covered in Module 3.1.

6 Where this sits in the track

Chinchilla reframed scaling from "how big a model can we afford?" to "how should we split a budget between size and data?". It is the quantitative backbone of every frontier-model training decision, including the GPT-4-era systems that motivate this module. Topic 2 derives the $C \approx 6ND$ law used here and extends the compute picture to *inference-time* scaling and *sparse* Mixture-of-Experts, where parameter count and FLOPs decouple. Module 3.1 (LLaMA) takes the inference-optimal turn flagged in the pitfall above.

A Reproducible (NumPy)

Inputs: fit constants $E = 1.69$, $A = 406.4$, $\alpha = 0.34$, $B = 410.7$, $\beta = 0.28$; the GPT-3 and Chinchilla (N, D) pairs; the 20:1 rule. The snippet reproduces every compute value, ratio, and exponent above.

```
import numpy as np
E,A,al,Bc,be = 1.69, 406.4, 0.34, 410.7, 0.28
# compute-optimal exponents of C
print(round(be/(al+be),3), round(al/(al+be),3))   # 0.452 0.548
def C(N,D): return 6*N*D
for name,N,D in [("GPT-3",175e9,300e9),("Chinchilla",70e9,1.4e12)]:
    print(name, "tok/param=", round(D/N,2), " C=%.3e"%C(N,D))
# GPT-3      tok/param= 1.71  C=3.150e+23
# Chinchilla tok/param= 20.0  C=5.880e+23
for N in [1e9,7e9,70e9]:                              # 20:1 table
    D=20*N; print("N=%.0e D=%.1e C=%.3e"%(N,D,C(N,D)))
# N=1e+09 D=2.0e+10 C=1.200e+20
# N=7e+09 D=1.4e+11 C=5.880e+21
# N=7e+10 D=1.4e+12 C=5.880e+23
```